Core 90 - Math and bit tricks

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Core 90 - Math and bit tricks | 1 questions | tap a title to jump to it

H 1. Check Digits and Combine Ranges **PracHub 2026-03-29**

STUDY PATH: question 90 of 167 (Core 90)

Priority score 3.44 | Math and bit tricks | asked 205 days ago

HARD | PracHub | Technical Screen | 1 of 1

★ TOP 75: rank 4 in the Medium and Hard priority list

★ Check Digits and Combine Ranges

POSTED 2026-03-29 | 205 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 73.6 | 30 DAYS 63.4 | PRIORITY 2/228

ABOUT THIS QUESTION

Topic: Core 90 - Math and bit tricks

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #9 Palindrome Number, LeetCode #56 Merge Intervals, LeetCode #228 Summary Ranges

LeetCode match: LC 9 Palindrome Number + LC 56 Merge Intervals / LC 228 Summary Ranges

Posted on PracHub: 2026-03-29 (updated 2026-05-19)

Original:
https://prachub.com/coding-questions/check-digits-and-combine-ranges

QUESTION

You are asked to solve two independent coding tasks.

Task 1:

Check Whether an Integer Reads the Same Backward Given an integer `x`, return `true` if its decimal representation reads the same from left to right and from right to left; otherwise return `false`. Requirements and edge cases:

- Negative integers should return `false` because of the leading minus sign.
- You may assume `x` fits in a signed 32-bit integer.
- Try to solve it without converting the integer to a string. Example:

```
Input: x = 121
Output: true
```

```
Input: x = -121
Output: false
```

```
Input: x = 10
Output: false
```

Task 2:

Combine Overlapping Ranges Given an array of intervals `intervals`, where each interval is represented as `[start, end]`, combine all overlapping intervals and return an array of non-overlapping intervals that covers exactly the same ranges. Requirements and edge cases:

- Intervals may be provided in any order.
- Two intervals overlap if they share at least one point. For example, `[1, 4]` and `[4, 5]` should be combined into `[1, 5]`.

QUESTION

- Return the intervals in increasing order by start value. Example:

```
Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
```

```
Input: intervals = [[1,4],[4,5]]
Output: [[1,5]]
```

Part 1: Check Whether an Integer Reads the Same BackwardGiven an integer x, return True if its decimal representation reads the same from left to right and from right to left. Otherwise, return False. Negative integers are not palindromes because of the leading minus sign. Try to solve the problem using arithmetic operations instead of converting the integer to a string.
Constraints-2^31 <= x <= 2^31 - 1Negative integers must return FalseAn arithmetic-based solution is preferred over string conversion

EXAMPLES

```
Input: 121
Expected Output: True
```

Explanation: 121 reads the same forward and backward.

```
Input: -121
Expected Output: False
```

Explanation: Negative numbers are not palindromes.

Hints

A negative number can never be a palindrome because the minus sign only appears on the left.Try reversing only the second half of the digits instead of the whole number.
Part 2: Combine Overlapping RangesGiven a list of intervals, where each interval is represented as [start, end], merge all overlapping intervals and return a new list of non-overlapping intervals that covers exactly the same ranges. Intervals may appear in any order. Two intervals overlap if they share at least one point, so [1, 4] and [4, 5] should be merged into [1, 5]. Return the result sorted by starting value.
Constraints
0 <= len(intervals) <= 10^4-10^9 <= start <= end <= 10^9Intervals may be unsorted
Examples
Input: [[1, 3], [2, 6], [8, 10], [15, 18]]
Expected Output: [[1, 6], [8, 10], [15, 18]]
Explanation: The first two intervals overlap and merge into [1, 6].
Input: [[1, 4], [4, 5]]
Expected Output: [[1, 5]]
Explanation: The intervals share the point 4, so they overlap.
Hints

Sort the intervals by their start value before trying to merge them.Keep track of the last interval in the output. If the next interval overlaps, extend the end; otherwise, start a new interval.

APPROACH

Task 1: reverse half the digits and compare (no string, handle negatives and trailing zeros). Task 2: sort ranges and merge or combine consecutive ones in one pass.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Two independent tasks. For the palindrome: negatives are false, no string conversion, 32-bit input. For the ranges: ask whether touching ranges such as [1, 4] and [4, 5] merge (the statement says they do) and whether adjacent integer ranges like [1, 2] and [3, 4] should also merge.

BRUTE FORCE TO SAY OUT LOUD

Palindrome: convert to a string and compare with its reverse. Ranges: compare every pair repeatedly until nothing merges, O(n^2).

OPTIMAL IDEA

Palindrome: return false for negatives and for any positive number ending in 0; then build a reversed number from the low digits while x > rev, and compare x with rev or rev // 10 (to skip the middle digit). Only half the digits are processed, so overflow cannot happen. Ranges: sort by start, then sweep, extending the last merged range when the next start is at most its end, else append. O(n log n).

DRY-RUN THESE CASES

121 true, -121 false, 10 false, 0 true, 1221, 12321. Ranges: [[1,3],[2,6],[8,10],[15,18]] gives [[1,6],[8,10],[15,18]]; [[1,4],[4,5]] gives [[1,5]]; unsorted input; one range; a range fully inside another.

LIKELY FOLLOW-UPS

Palindrome in another base, or palindrome linked list. Ranges: insert a new interval into a sorted list (LeetCode 57), summary ranges of sorted integers (LeetCode 228).

PITFALLS

Reversing the whole number and overflowing. Forgetting to sort before merging. Using strict less-than when touching ranges must merge.

HOW TO THINK ABOUT IT

Task 1: a number is a palindrome if its second half reversed equals its first half, so reverse only half the digits (stop when the reversed part catches up) and avoid overflow. Negatives and numbers ending in 0 are never palindromes. Task 2: after sorting by start, an overlap can only be with the last merged range, so one pass with a 'current' range is enough.

Basis: Two independent tasks: LeetCode 9 (Palindrome Number, no string conversion) and LeetCode 56 (Merge Intervals; ranges sharing a point merge).

BRUTE FORCE · Task 1: convert to a string and compare with its reverse

BRUTE FORCE · PSEUDO-CODE

```
def is_palindrome(x):
    s = str(x)
    return x >= 0 and s == s[::-1]
```

TIME	SPACE
O(d)	O(d)

WHY IT IS SLOW

O(d) time but it builds strings, which the question forbids. Reversing only the second half of the digits arithmetically (x % 10, x // 10) uses O(1) space.

BRUTE FORCE · Task 2: merge any two overlapping ranges until none overlap

BRUTE FORCE · PSEUDO-CODE

```
def combine(ranges):
    ranges = list(ranges)
    merged = True
    while merged:
        merged = False
        for a, b in combinations(ranges, 2):
            if overlaps(a, b):
                ranges.remove(a)
                ranges.remove(b)
                ranges.append(union(a, b))
                merged = True
                break
    return sorted(ranges)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each pass compares all pairs, O(n^2), and several passes may be needed. Sorting by start and sweeping once is O(n log n).

OPTIMAL SOLUTION · Task 1: reverse only the second half arithmetically

TIME	SPACE
O(d)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task 2: sort by start, then one sweep

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Adjacent integer ranges ([1, 2] and [3, 4]) should merge too.

Change the overlap test to s <= out[-1][1] + 1 (integers only). For real-valued ranges that is wrong, so say which you assume.

FOLLOW-UP 2 Summarise a sorted list of integers as ranges (LeetCode 228).

One pass keeping the current range and extending it while the next number is exactly one more; the same 'extend or start new' structure as the merge.

FOLLOW-UP 3 Palindrome in another base, or as a linked list.

Another base replaces 10 by the base in % and //. For a linked list find the middle with slow and fast pointers, reverse the second half in place and compare.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Check Digits and Combine Ranges
#
# You are asked to solve two independent coding tasks. ### Task 1: Check Whether an Integer Reads the Same Backward
# Given an integer x, return true if its decimal
# representation reads the same from left to right and
# from right to left; otherwise return false. Requirements
# and edge cases: - Negative integers should return false
# because of the leading minus sign. - You may assume x
# fits in a signed 32-bit integer. - Try to solve it
# without converting the integer to a string. Example:
# text Input: x = 121 Output: true text Input: x = -121
# Output: false text Input: x = 10 Output: false ### Task
# 2: Combine Overlapping Ranges Given an array of
# intervals intervals, where each interval is represented
# as [start, end], combine all overlapping intervals and
# return an array of non-overlapping intervals that covers
# exactly the same ranges.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Adjacent integer ranges ([1, 2] and [3, 4]) should
# merge too.
# 2. Summarise a sorted list of integers as ranges
# (LeetCode 228).
# 3. Palindrome in another base, or as a linked list.
#
# INTERVIEW NOTES
# Task 1: a number is a palindrome if its second half
# reversed equals its first half, so reverse only half the
# digits (stop when the reversed part catches up) and
# avoid overflow. Negatives and numbers ending in 0 are
# never palindromes. Task 2: after sorting by start, an
# overlap can only be with the last merged range, so one
# pass with a 'current' range is enough.
#
# BRUTE FORCE (say it first): Task 1: convert to a string
# and compare with its reverse
# str(x) == str(x)[::-1] after rejecting negatives.
# Correct and short; the question asks to avoid the
# string. Time O(d). Space O(d).
```

```
# BRUTE FORCE (say it first): Task 1: convert to a string
# and compare with its reverse | time O(d)
def palindrome_number_string(x):
    return x >= 0 and str(x) == str(x)[::-1]


# BASE QUESTION (optimal): reverse only the second half
# arithmetically
# Idea: Reject negatives and positive numbers ending in 0.
def is_palindrome(x, base=10):
    # no string: build the reversed second half
    if x < 0 or (x % base == 0 and x != 0):
        return False
    rev = 0
    # stop when the reversed half catches up with the rest
    while x > rev:
        rev = rev * base + x % base
        x //= base
    # an odd number of digits has a middle digit to drop
    return x == rev or x == rev // base


# TASK 2: combine ranges
def merge_ranges(intervals, adjacent=False):
    out = []
    # sort by start so overlaps are neighbours
    for s, e in sorted(intervals):
        # sharing one point counts as overlap: [1,4] +
        # [4,5]
        gap = 1 if adjacent else 0
        if out and s <= out[-1][1] + gap:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out


# FOLLOW-UP 2: Summarise a sorted list of integers as
# ranges (LeetCode 228).
# Answer: One pass keeping the current range and extending
# it while the next number is exactly one more; the same
# 'extend or start new' structure as the merge.
def summarize_ranges(nums):
    # LeetCode 228: a sorted list of ints as ranges
    out = []
    for x in nums:
        if out and x == out[-1][1] + 1:
            # exactly one more: extend the current range
            out[-1][1] = x
        else:
            out.append([x, x])
    return out


if __name__ == "__main__":
    assert is_palindrome(121) and is_palindrome(0)
    assert not is_palindrome(-121)
    assert not is_palindrome(10)
    assert is_palindrome(1221) and not is_palindrome(123)
    # binary 101 is 5; binary 110 is 6
    assert is_palindrome(5, 2) and not is_palindrome(6, 2)
    ranges = [[1, 3], [2, 6], [8, 10], [15, 18]]
    merged = merge_ranges(ranges)
    assert merged == [[1, 6], [8, 10], [15, 18]]
    assert merge_ranges([[1, 4], [4, 5]]) == [[1, 5]]
    # any input order, and an empty list
    shuffled = merge_ranges([[8, 10], [1, 3]])
    assert shuffled == [[1, 3], [8, 10]]
    assert merge_ranges([]) == []
```

```
    # adjacent integer ranges merge only when asked
    touching = [[1, 2], [3, 4]]
    assert merge_ranges(touching) == touching
    assert merge_ranges(touching, True) == [[1, 4]]
    assert summarize_ranges([0, 1, 2, 4, 5, 7]) == [
        [0, 2], [4, 5], [7, 7]]
```

FUNCTION AND TEST CASES

Signature

```
def solution(x: int) -> bool:
SAMPLE INPUT
121
```

Test cases

Input	Expected
121	True
-121	False